



Procedures

As our code grows in complexity, we need to start refactoring our code to make more efficient use of the instructions and make it easier to read and understand. A common way to do so is through the use of `functions` and `procedures`. While functions require a calling procedure to call them and pass their arguments (as we will discuss in the next section), `procedures` are usually more straightforward and mainly used for code refactoring.

A `procedure` (sometimes referred to as a `subroutine`) is usually a set of instructions we want to execute at specific points in the program. So instead of reusing the same code, we define it under a procedure label and `call` it whenever we need to use it. This way, we only need to write the code once but can use it multiple times. Furthermore, we can use procedures to split a larger and more complex code into smaller, simpler segments.

Let's go back to our code:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a
```

nasm

```

section .text
_start:
    mov rax, 1          ; rax: syscall number 1
    mov rdi, 1          ; rdi: fd 1 for stdout
    mov rsi,message     ; rsi: pointer to message
    mov rdx, 20         ; rdx: print length of 20 bytes
    syscall             ; call write syscall to the intro message
    xor rax, rax        ; initialize rax to 0
    xor rbx, rbx        ; initialize rbx to 0
    inc rbx             ; increment rbx to 1

loopFib:
    add rax, rbx        ; get the next number
    xchg rax, rbx       ; swap values
    cmp rbx, 10         ; do rbx - 10
    js loopFib          ; jump if result is <0
    mov rax, 60
    mov rdi, 0
    syscall

```

We see that we are now doing multiple things in a big chunk of code:

1. Printing the intro message
2. Setting initial Fibonacci values to `0` and `1`
3. Using a loop to calculate the following Fibonacci number
4. Exiting the program

Our loop is already defined under a label, so we can call it when we need it. However, the three other parts of the code can be refactored as procedures to call them whenever we need to, increasing code efficiency.

Defining Procedures

As a starting point, let's add a label above each of the three parts of the code we want to turn into procedures:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a

section .text
_start:

printMessage:
    mov rax, 1      ; rax: syscall number 1
    mov rdi, 1      ; rdi: fd 1 for stdout
    mov rsi, message ; rsi: pointer to message
    mov rdx, 20     ; rdx: print length of 20 bytes
    syscall         ; call write syscall to the intro message

initFib:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1

loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    cmp rbx, 10     ; do rbx - 10
    js loopFib      ; jump if result is <0
```

nasm

```
Exit:
    mov rax, 60
    mov rdi, 0
    syscall
```

We see that our code already looks better. However, this is not any more efficient than it was, as we could have achieved the same by using comments. So, our next step is to use `calls` to direct the program to each of our procedures.

CALL/RET

When we want to start executing a procedure, we can `call` it, and it will go through its instructions. The `call` instruction pushes (i.e., saves) the next instruction pointer `rip` to the stack and then jumps to the specified procedure.

Once the procedure is executed, we should end it with a `ret` instruction to return to the point we were at before jumping to the procedure. The `ret` instruction `pops` the address at the top of the stack into `rip`, so the program's next instruction is restored to what it was before jumping to the procedure.

The `ret` instruction plays an essential role in **Return-Oriented Programming (ROP)**, an exploitation technique usually used with Binary Exploitation.

Instruction	Description	Example
<code>call</code>	push the next instruction pointer <code>rip</code> to the stack, then jumps to the specified procedure	<code>call printMessage</code>
<code>ret</code>	pop the address at <code>rsp</code> into <code>rip</code> , then jump to it	<code>ret</code>

So with that, we can set up our calls at the beginning of our code to define the execution flow we want:

```
global _start

section .data
    message db "Fibonacci Sequence:", 0x0a

section .text
_start:
    call printMessage    ; print intro message
    call initFib         ; set initial Fib values
    call loopFib         ; calculate Fib numbers
    call Exit            ; Exit the program

printMessage:
    mov rax, 1           ; rax: syscall number 1
    mov rdi, 1           ; rdi: fd 1 for stdout
    mov rsi, message     ; rsi: pointer to message
    mov rdx, 20          ; rdx: print length of 20 bytes
    syscall              ; call write syscall to the intro message
    ret
```

nasm

```

initFib:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
    ret

loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    cmp rbx, 10     ; do rbx - 10
    js loopFib      ; jump if result is <0
    ret

Exit:
    mov rax, 60
    mov rdi, 0
    syscall

```

This way, our code should execute the same instructions as before while having our code cleaner and more efficient. From now on, if we need to edit a specific procedure, we won't have to display the entire code, but only that procedure. We can also see that we did not use `ret` in our `Exit` procedure, as we don't want to return to where we were. We want to exit the code. We will almost always use a `ret`, and the `Exit` function is one of the few exceptions.

Note: It is important to understand the line-based execution flow of assembly. If we don't use a `ret` at the end of a procedure it will simply execute the next line. Likewise, had we returned at the end of our `Exit` function, we would simply go back and execute the next line, which would be the first line of `printMessage`.

Finally, we should also mention the `enter` and `leave` instructions, which are sometimes used with procedures to save and restore the addresses of `rsp` and `rbp` and allocate a specific stack space to be used by the procedure. We won't be needing to make use of them in this module, however.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

+1

+40



Try assembling and debugging the above code, and note how "call" and "ret" store and retrieve "rip" on the stack. What is the address at the top of the stack after entering "Exit"? (6-digit hex 0xaddress, without zeroes)

Write your answer

Submit

Hint

← Previous

Section 18 / 24

 +20

Next

Mark Complete & Next